

A melhor otimização foi a que decidi *não fazer.*

Construí um agente que lê a caixa de apoio ao cliente de uma loja online e escreve rascunhos. A parte difícil não foi ligar um LLM ao email.

427

emails processados em produção

0

enviados sem revisão humana

67%

escalados — quase todos com razão

\$0,008

o teste que travou um deploy

Todos os números verificados contra a base de dados de produção e o código a 3 de setembro de 2026. Onde não existe medição, está escrito que não existe.

Um LLM resolve isto num playground. Em produção, não.

Uma loja online recebe correspondência o dia inteiro: estado de encomendas, devoluções, defeitos, trocas, reembolsos, queixas. Cada resposta exige três coisas ao mesmo tempo — conhecer a política da loja, ter os dados daquela encomenda à frente, e escrever num tom consistente.

A resposta imediata é ligar um modelo à caixa de correio. Funciona: o modelo escreve emails de apoio ao cliente perfeitamente credíveis.

O problema é que a maior parte destas mensagens **não deve** ser respondida automaticamente. Reembolsos movem dinheiro. Alterar uma encomenda exige escrita em sistemas que não devem estar ao alcance de um modelo. E responder com confiança a uma política que não se conhece é pior do que não responder — um cliente com uma promessa errada é um problema maior do que um cliente que espera.

A PERGUNTA DE DESENHO

A pergunta nunca foi como automatizar o apoio ao cliente. Foi *o que é seguro automatizar, e como é que o sistema sabe a diferença.*

O modelo faz uma coisa só.

Corre de dois em dois minutos. Para cada email decide entre três ações. Tudo o que pode ser decidido em código é decidido **antes** de o modelo ver o caso.

ARQUITETURA

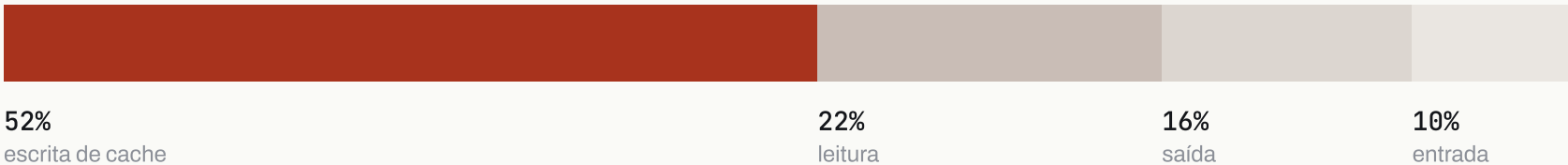


Provar identidade, somar prazos, descartar ruído e validar a decisão vivem em código. Cada uma foi para lá **porque o modelo demonstrou executá-la mal em produção** — a fronteira foi movida com evidência, não desenhada à partida.

A taxa de cache parecia ótima.

O registo não guardava tokens nenhuns. A única fonte era a fatura mensal — agregada, impossível de atribuir a um email. Antes de otimizar o que quer que fosse, instrumentei: sete colunas novas no registo.

DECOMPOSIÇÃO DO CUSTO DE UM DIA



CUSTO

89% de acerto de cache parecia resolvido. Mas escrever custa um múltiplo do que custa ler — *os 11% de falhas custavam 2,4× mais do que todos os acertos somados.*

\$0,0135

ler o prefixo em cache

\$0,248

reescrevê-lo num arranque a frio

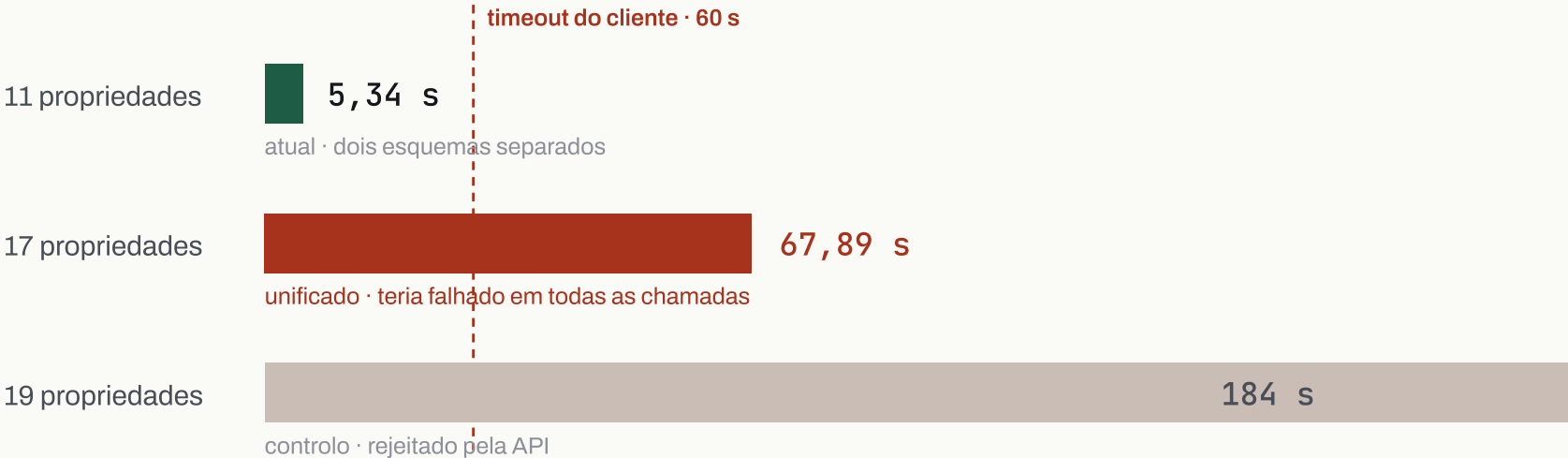
O problema nunca foi o prompt ser grande. Era o número de vezes que era reescrito.

Unificar parecia obviamente melhor.

A investigação revelou algo que eu não sabia: **havia duas entradas de cache, não uma**. O esquema de saída faz parte do prefixo, por isso as duas chamadas nunca a partilhavam — um arranque a frio numa escalação reescrevia o prompt inteiro duas vezes.

A hipótese era evidente: um esquema só, uma entrada só, metade da escrita desaparecia.

O TESTE — UMA CHAMADA REAL POR CONFIGURAÇÃO



DECISÃO · REJEITADA Teria estourado o timeout em todas as chamadas — falha total, não intermitente.	CUSTO NÃO LINEAR 11 props → 5 s, 17 → 68 s. Aparar um campo ou dois também não salvava.	CUSTO DO TESTE \$0,008
--	---	--

Uma otimização rejeitada depois de um teste *é um bom resultado de engenharia.*

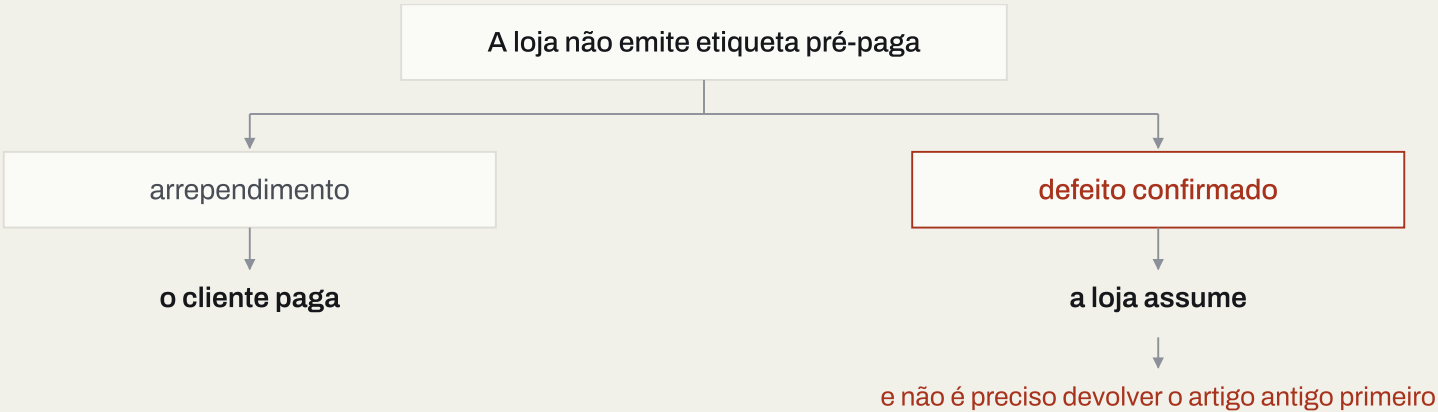
Epílogo: as duas entradas de cache **deixaram de existir a 1 de setembro** — não por unificar os esquemas, mas por remover a segunda chamada por completo. O mesmo problema, resolvido pela via que a medição não tinha proibido.

Uma pergunta simples não é uma regra simples.

A base de conhecimento tem 1 088 linhas em 7 ficheiros e é a única fonte que o modelo pode citar. Escrevê-la foi a parte do projeto que mais tempo consumiu — e a que menos se parece com programação.

PERGUNTA DO CLIENTE

“Quem paga o envio da devolução?”



Duas regras verdadeiras sobre o mesmo assunto, que se contradizem lidas fora de contexto. Tiveram de ficar explicitamente ligadas uma à outra — cada uma a apontar para a outra — para o modelo não aplicar a errada com confiança.

Uma base ambígua não produz respostas erradas. *Produz escalações a mais* — porque o agente é instruído a escalar na dúvida.

O custo da ambiguidade aparece na fatura e na carga de trabalho, não na qualidade.

67% escalavam. Quis baixar o número.

Cada escalação custa mais do que um rascunho. A leitura fácil é “há aqui muito para automatizar”. Passei um dia a ler os motivos, um a um.

CATEGORIA	N	PORQUE ESCALA
Ação sobre encomenda	98	exige escrita na plataforma de comércio
Compromisso anterior	89	estado que não existe em nenhum sistema legível
Julgamento humano	51	garantia, litígio, exceção, gesto comercial
Outras seis categorias	46	contexto, identidade, lacuna, inventário

MEDIDO

Testei a hipótese mais promissora — “as perguntas de reembolso podiam ser respondidas com dados que a plataforma já tem”. Fui buscar as encomendas envolvidas.

2/9

tinham o reembolso
registado. Nas outras 7
estava mesmo por
processar

Baixar aquela percentagem exigia dar ao agente escrita na plataforma — *o que apaga o guardrail que torna o sistema tolerável.*

Um guardrail de prompt contorna-se. Um de infraestrutura não.

22

guardrails documentados, em quatro níveis

MAIS FRACO

Prompt10

2

Esquema

Código7

3

Infraestrutura

MAIS FORTE

Sem Mail.Send

A aplicação nunca pediu a permissão de envio

Sem escrita na loja

Apenas leitura de encomendas

Revisão obrigatória

Todo o output passa por uma pessoa

Mesmo que os vinte guardrails anteriores falhem em simultâneo, o pior resultado possível é *um rascunho errado que uma pessoa lê e apaga.*

Nenhum email sai. Nenhuma encomenda muda.

Sete dos vinte e dois existem porque algo correu mal.

O QUE ACONTECEU

O modelo errou o cálculo de um prazo
com a data de entrega correta no contexto

Formulários do site descartados
pedidos reais apanhados como ruído automático

Uma paragem total sem alerta nenhum
a conta ficou sem saldo; o sistema saía com código de sucesso

O deploy disse “Enviado” sem enviar
o código de saída não apanhou a falha

Publicar uma regra custa dinheiro
a base faz parte do prefixo em cache; cada publicação invalida-o

O QUE MUDOU

O cálculo passou para Python e chega ao modelo já pronto

Exceções explícitas que confirmam o conteúdo antes de descartar

Contagem de passagens consecutivas sem decisão produzida

O script confirma no servidor que o conteúdo chegou

O deploy compara uma impressão digital do prompt e avisa se aquela publicação é gratuita — informativo, nunca bloqueante

INCIDENTES

O padrão comum: nenhum foi encontrado a ler código. Foram todos encontrados a comparar o que o sistema produziu com o que realmente aconteceu a seguir.

O sinal de maturidade não é ter guardrails — é a proveniência deles. Cada um tem data e caso de origem registados no repositório.

Os números, e o que eles não provam.

VOLUME	CUSTO	SEGURANÇA	TESTES
427 emails processados	\$0,0317 por email	0 emails enviados sem revisão	305 testes unitários
25 · 67 · 8 % rascunho · escalção · descarte	92% do prefixo lido da cache	0 alterações a encomendas	98 casos no banco de ensaio

QUALIDADE — O RASCUNHO CONTRA O QUE A PESSOA REALMENTE ENVIOU

160

casos comparáveis

100%

semelhança mediana, em 160 casos

31

divergiram mais de 10% — cada uma virou regra nova ou caso de ensaio

A RESSALVA FAZ PARTE DO RESULTADO

Não existe linha de base limpa de custo. A telemetria é mais recente do que o sistema, e cobre 228 dos 427 emails — comparar “antes e depois” não é honesto.

A poupança do aquecedor de cache não tem amostra. Uma noite: 8 arranques a frio passaram a 0. Uma noite não é uma medição.

Nada verifica contradições na base de conhecimento de forma contínua, e a saliência continua por resolver — regras corretas que o modelo por vezes não aplica.

Instrumentar antes de otimizar não é um slogan.

As duas maiores otimizações foram invisíveis até haver colunas de custo no registo.

Uma média pode esconder uma assimetria de preço.

89% parecia resolvido. Os 11% restantes custavam mais do que todos os acertos juntos.

Uma hipótese rejeitada por medição é um resultado.

Medir custa quase sempre menos do que reverter um deploy.

Knowledge engineering é engenharia por direito próprio.

Condicionais, exceções, regras com prazo de validade, e saliência.

Nem toda a automação por fazer é dívida.

Ler os motivos de escalação um a um mostrou que quase todos estavam certos.

A métrica certa não era quantos casos automatizei. Era *quantos consegui automatizar com segurança.*

STACK

Python 3.14 · SQLite · systemd · Microsoft Graph ·
Shopify Admin API (leitura) · Anthropic Messages API ·
unittest

DELIBERADAMENTE AUSENTES

Sem framework web, ORM, fila de mensagens, contentores ou framework de agentes — é um sistema que **uma pessoa sozinha** tem de conseguir manter.

Loja, domínios, endereços, identificadores de encomenda e nomes de pessoas foram omitidos em todo o documento.